



UNIVERSITÉ DE LIMOGES
FACULTÉ DES SCIENCES ET TECHNIQUES

UE Crypto avancée

Rapport de projet :
Cryptographie basé sur les codes

Réalisé par :

Karim Mohamed Cherif RAHMANI (*Master CRYPTIS Mathématiques*)

Mohamed Rami BOUAROUR (*Master CRYPTIS Informatique*)

Année universitaire : 2025-2026

Table des matières

1	Algorithme ISD (<i>Information Set Decoding</i>)	2
1.1	Principe de l'algorithme ISD	2
1.1.1	Les étapes	2
1.1.2	Complexité ISD	2
1.2	Inversion de matrice binaire par la méthode du pivot de Gauss	2
1.2.1	Étapes	3
1.2.2	Implémentation	3
1.3	Implémentation de l'algorithme ISD	3
2	MDPC (Moderated Density Parity Check)	5
2.1	Algorithme BitFlip	5
2.1.1	Étapes	5
2.2	Système de chiffrement MDPC	5
2.2.1	Génération des clés	5
2.2.2	Chiffrement	6
2.2.3	Déchiffrement	6
2.3	Implémentation de L'algorithme MDPC	6
2.3.1	Variation du seuil T	7
2.3.2	Variation du poids W	8

1 Algorithme ISD (*Information Set Decoding*)

1.1 Principe de l'algorithme ISD

ISD est un algorithme de décodage par syndrome, qui cible les algorithmes du chiffrement basés sur les codes. il utilise une approche aléatoire, ou il fait des permutations aléatoires en espérant tomber sur une partition favorable ou toutes les erreurs se trouvent dans la partie redondance.

1.1.1 Les étapes

Entrée : La matrice de contrôle H , et le syndrome s , et un poids T

Sortie : Une erreur e ; tq $H \times e^T = s$ et $w(e) \leq T$

1. **Permutation** On applique une permutation aléatoire P sur les colonnes.
2. **Application du pivot de Gauss :** On applique le pivot de Gauss sur les $n - k$ colonnes de H afin de la mettre sous forme systématique.

$$H \xrightarrow{\text{Gauss}} [A \mid I_{n-k}]$$

Les mêmes opérations élémentaires sont appliquées simultanément au syndrome s , ce qui nous donne s' .

3. **Vérification :** Si $w(s') = T$, alors s' correspond directement au vecteur erreur sur les positions de redondance tandis que celle de gauche sont nulle. et on reconstruit e en appliquant la permutation inverse P^{-1} sur e' .

$$e = P^{-1}((0, 0, \dots, 0), (s'))$$

Sinon, on retourne à l'étape 1 et on essaie une nouvelle permutation.

1.1.2 Complexité ISD

La complexité de l'ISD dépend de : le coût des opérations matricielles effectuées à chaque itération et le nombre moyen d'itérations nécessaires pour trouver une bonne permutation.

- **Coût d'une itération :** L'opération la plus coûteuse de la boucle est l'application du pivot de Gauss sur la matrice, dont le coût est évalué à $\mathcal{O}((n - k)^3)$ opérations
- **Probabilité de succès :** L'algorithme ne s'arrête que si la permutation choisie place l'intégralité des T erreurs dans la partie redondante de la matrice. La probabilité de tirer une telle permutation au hasard est donnée par : $\frac{\binom{n-T}{k}}{\binom{n}{k}} = \frac{\binom{n-k}{T}}{\binom{n}{T}}$.

Le nombre moyen d'itérations correspond à l'inverse de cette probabilité. La complexité de l'algorithme ISD est donc :

$$\mathcal{O} \left((n - k)^3 \frac{\binom{n}{T}}{\binom{n-k}{T}} \right)$$

C'est cette complexité exponentielle est ce qui garantit la sécurité des cryptosystèmes basés sur les codes face à ce type d'attaque.

1.2 Inversion de matrice binaire par la méthode du pivot de Gauss

L'inversion d'une matrice binaire M de taille $n \times n$ sur $\mathbb{F}_2$ s'effectue par la méthode de Gauss-Jordan. Le principe est de construire la matrice augmentée $[M \mid I_n]$ et d'appliquer des opérations élémentaires sur les lignes jusqu'à obtenir $[I_n \mid M^{-1}]$.

1.2.1 Étapes

Entrée : Une matrice binaire carrée M de taille $n \times n$

Sortie : La matrice inverse M^{-1} si elle existe, sinon on retourne que la matrice est non inversible.

1. **Construction de la matrice augmentée :** On construit la matrice $[M \mid I_n]$ de taille $n \times 2n$.
2. **Pivot de Gauss-Jordan :** Pour chaque colonne i , on cherche un pivot sur la diagonale. Si le pivot est nul, on cherche une ligne $j > i$ ayant un 1 en colonne i et on échange les deux lignes, tel que l'élément de cette colonne qui fait partie de la diagonale soit un 1. Si aucun pivot n'est trouvé, la matrice est non inversible et on retourne faux. Une fois le pivot trouvé, on élimine les 1 sur toutes les autres lignes par XOR :

$$\text{ligne}_j = \text{ligne}_j \oplus \text{ligne}_i \quad \forall j \neq i \text{ tel que } M2[j][i] = 1$$

3. **Extraction de l'inverse :** Une fois la partie gauche réduite à I_n , la partie droite de la matrice augmentée contient M^{-1} .

1.2.2 Implémentation

```

karim@PCKarim:~/mnt/c/users/rahma/desktop$ ./inv_guauss
On va inverser la matrice suivante
1 1 0 1 1 1 0 0 0
0 1 0 1 0 1 1 0 1
1 0 1 0 0 1 1 1 0
1 0 1 1 1 0 1 1 0
0 1 0 0 1 0 0 0 0
1 1 1 0 1 1 1 1 0
0 0 1 1 0 0 0 1 1
1 0 0 1 0 1 1 1 1
1 1 1 0 1 1 1 1 0
Matrice non inversible
karim@PCKarim:~/mnt/c/users/rahma/desktop$ ./inv_guauss
On va inverser la matrice suivante
1 0 1 0 1 0 1 1 1
1 1 1 1 0 1 0 1 1
0 1 1 1 0 1 1 0 0
1 0 1 0 1 0 0 1 1
0 1 1 1 0 0 0 0 0
1 0 1 0 0 0 1 1 0
0 0 1 1 0 1 0 0 0
0 0 1 0 0 0 1 1 1
1 0 1 1 1 1 0 0 1
Voici l'inverse de la matrice M
1 0 0 0 0 1 1 0 1
1 0 1 1 0 0 1 0 0
1 1 1 0 0 1 1 1 1
0 1 0 1 1 1 0 1 1
0 0 0 0 0 1 1 1 1
1 0 1 1 1 0 0 0 0
1 0 0 1 0 0 0 0 0
1 1 1 1 0 1 0 1 0
1 0 0 0 0 0 1 1 1

```

FIGURE 1 – Inversion de matrice 9×9

1.3 Implémentation de l'algorithme ISD

Pour l'implémentation de l'algorithme ISD, afin d'optimiser les temps de calcul face à la complexité exponentielle de l'algorithme, On a parallélisé le code pour accélérer toutes les opérations

effectué par l'algorithme.

Le programme effectue une boucle continue où, à chaque tentative, une permutation aléatoire est appliquée aux colonnes de la matrice de contrôle H . Une élimination de Gauss est ensuite appliquée pour mettre la matrice sous forme systématique $[A \mid I_k]$

Si A est inversible, la même transformation est appliquée au syndrome s . Si le poids de ce nouveau syndrome est inférieur ou égal au seuil T ($w(s') \leq T$), cela signifie que nous avons trouvé la permutation voulue. L'erreur est alors reconstruite à partir de ce syndrome.

Résultat d'application pour les paramètres du TP

1. $N = 400$, $K = 200$, $T = 20$

[illegible]FIGURE 2 – Résultats d'application pour $N = 400$, $K = 200$, $T = 20$

Pour les paramètres $N = 400$, $K = 200$, $T = 20$, on arrive a retrouver l'erreur avec succès en $2815 \text{ secondes} \approx 47 \text{ minutes}$.

Vu la difficulté de ces paramètres, on peut considérer le temps consommé par notre algorithme comme raisonnable.

2. $N = 1000$, $K = 500$, $T = 10$

[illegible]FIGURE 3 – Résultats d'application pour $N = 1000$, $K = 500$, $T = 10$

Pour les paramètres $N = 1000$, $K = 500$, $T = 10$, on arrive à retrouver l'erreur avec succès en $489 \text{ secondes} \approx 8 \text{ minutes}$

2 MDPC (Moderated Density Parity Check)

MDPC est un type de code correcteur, il est utilisé pour le chiffrement et il est considéré comme une version améliorée de McEliece. En principe elle a deux polynômes x et y , qui sont denses, et qu'ils forment la clé privée comme suite : $h = x^{-1} \times y$.

Le chiffrement sera sous la forme $c = m \times h + e$.

2.1 Algorithme BitFlip

L'algorithme BitFlip est un algorithme de décodage itératif qui corrige les erreurs dans les codes MDPC. Son principe est que pour chaque bit reçu, on compte combien d'équations de parité non satisfaites il implique, et si ce nombre dépasse un seuil T , on retourne ce bit ($0 \rightarrow 1$ ou $1 \rightarrow 0$).

2.1.1 Étapes

Entrée : Le mot reçu (c_0, c_1) , les clés privées x et y , un seuil T

Sortie : (c_0, c_1) corrigé tel que $w(s) = 0$, $\perp$ sinon.

1. **Calcul du syndrome :** On calcule le syndrome s à partir du mot reçu et des clés privées, où la multiplication est polynomiale circulaire dans $\mathbb{F}_2[X]/(X^N - 1)$:

$$s = c_0 \cdot x + c_1 \cdot y$$

Si $w(s) = 0$, il n'y a pas d'erreur et on s'arrête.

2. **Comptage :** Pour chaque bit i , on compte combien de bits du syndrome valent 1 et impliquent ce bit :

$$\text{cpt}_0[i] = \#\{j \mid s[j] = 1 \text{ et } x[(j - i \bmod N)] = 1\}$$

$$\text{cpt}_1[i] = \#\{j \mid s[j] = 1 \text{ et } y[(j - i \bmod N)] = 1\}$$

3. **Flip des bits :** Tout bit impliqué dans plus de T erreurs reçoit $\oplus 1$:

$$c_0[i] \leftarrow c_0[i] \oplus 1 \quad \text{si } \text{cpt}_0[i] \geq T$$

$$c_1[i] \leftarrow c_1[i] \oplus 1 \quad \text{si } \text{cpt}_1[i] \geq T$$

On répète les étapes 1, 2 et 3 jusqu'à ce que $w(s) = 0$ ou qu'on atteigne le nombre maximum d'itérations.

2.2 Système de chiffrement MDPC

MDPC (Moderate Density Parity Check) est un système de chiffrement basé sur les codes correcteurs, il est basé sur McEliece. Il repose sur deux polynômes creux x et y dans $\mathbb{F}_2[X]/(X^N - 1)$ qui constituent la clé privée.

2.2.1 Génération des clés

On génère aléatoirement deux polynômes x et y de poids w . On calcule ensuite la clé publique g :

$$g = x \cdot y^{-1} \quad \text{dans } \mathbb{F}_2[X]/(X^N - 1)$$

2.2.2 Chiffrement

On génère deux vecteurs d'erreur e_0 et e_1 de poids total E ; tel que le nombre d'erreurs est distribué aléatoirement sur e_0 et e_1 , et on calcule :

$$c_0 = m + e_0$$

$$c_1 = m \cdot g + e_1$$

2.2.3 Déchiffrement

Le déchiffrement utilise la clé privée (x, y) . On calcule d'abord le syndrome :

$$s = c_0 \cdot x + c_1 \cdot y$$

On applique ensuite l'algorithme BitFlip pour retrouver les erreurs e_0 et e_1 , et on récupère le message :

$$m = c_0 \oplus e_0$$

2.3 Implémentation de L'algorithme MDPC

Pour l'implémentation du système MDPC, dans le but d'optimiser le code on a opté pour la multiplication circulaire des polynômes dans $\mathbb{F}_2[X]/(X^N - 1)$, où les polynômes sont représentés comme des tableaux binaires de taille N . Quant à l'inversion du polynôme on utilise l'algorithme d'Euclide étendu. (Si y n'est pas inversible, on en génère un nouveau jusqu'à en trouver un inversible). La clé publique est ensuite calculée par $g = x \cdot y^{-1}$. Les vecteurs d'erreur e_0 et e_1 sont générés aléatoirement avec un poids total de $E = 78$, en garantissant au moins une erreur dans chaque bloc. L'algorithme BitFlip est limité à 100 itérations maximum pour éviter une boucle infinie. Le message est ensuite récupéré par $m = c_0 \oplus e_0$.

Résultat pour les paramètres du TP

$$w = 39, N = 4813, E = 78, T = 26$$

```

Génération des clés:
Temps pris pour la Génération des clés est 0.327089:

Chiffrement:
Temps pris pour le chiffrement est 0.032144:

Déchiffrement:
BitFlipping réussie en 3 iterations
Temps pris pour le déchiffrement est 0.468205:

Le message est déchiffré avec succès

```

FIGURE 4 – Application des paramètres du TP

Pour les calculs c'est instantanée pour toutes les opérations, on peut voir sur le résultat que ça prend moins d'une seconde que ce soit pour le chiffrement, déchiffrement ou la génération des clés.

2.3.1 Variation du seuil T

Augmentation du seuil T

Lorsque on augmente le seuil de vers 27 et 28 l'algorithme fonctionne et arrive a déchiffrer le message mais il fait plus d'itérations Bitflipping et prend plus de temps.

```
Génération des clés:  
Temps pris pour la Génération des clés est 0.342511:  
  
Chiffrement:  
Temps pris pour le chiffrement est 0.033364:  
  
Déchiffrement:  
BitFlipping réussie en 7 iterations  
Temps pris pour le déchiffrement est 1.031679:  
  
Le message est déchiffré avec succès
```

FIGURE 5 – Résultat pour $T = 28$

Lorsque on l'augmente vers 29 ça déchiffre rarement, et quand ça fonctionne ça prend beaucoup plus d'itérations et plus de temps.

A partir de 30 le programme ne fonctionne plus.

Diminution du seuil T

Lorsque on Diminue le seuil de vers 25, 24 et 23 l'algorithme fonctionne mieux et arrive a déchiffrer le message en effectuant moins d'itérations bitflipping.

```
Génération des clés:  
Temps pris pour la Génération des clés est 0.358524:  
  
Chiffrement:  
Temps pris pour le chiffrement est 0.037408:  
  
Déchiffrement:  
BitFlipping réussie en 2 iterations  
Temps pris pour le déchiffrement est 0.413014:  
  
Le message est déchiffré avec succès
```

FIGURE 6 – Résultat pour $T = 23$

Lorsque on diminue vers 22 ça arrive a déchiffrer rarement et ça prend plus de temps.

A partir de 21 l'algorithme n'arrive plus a déchiffrer.

Donc pour la variations du seuil T on peut l'augmenter et le diminuer ça fonctionnera mais juste aux alentours de 26, a partir du moment ou on s'éloigne du seuil avec plus ou moins 2 le déchiffrement ne fonctionne plus.

Si le déchiffrement fonctionne, lorsque on augmente ça prend plus de temps, alors que lorsque on diminue ça prend moins de temps. La raison de cette symétrie est qu'un seuil très bas retourne plus de bits à chaque itération ce qui accélère le déchiffrement. Quant à un seuil plus haut

retourne moins de bits à chaque itérations ce qui ralentit le déchiffrement jusqu'à l'échec du déchiffrement.

2.3.2 Variation du poids W

Lorsqu'on augmente w de 10, le déchiffrement échoue.

```
karim@PCKarim:/mnt/c/users/rahma/desktop$ ./MDPC3
Génération des clés:
Temps pris pour la Génération des clés est 0.530640:

Chiffrement:
Temps pris pour le chiffrement est 0.029479:

Déchiffrement:
BitFlipping échoué
Temps pris pour le déchiffrement est 22.373188:

Echec de déchiffrement
```

FIGURE 7 – Augmentation de w de 10 $w = 49$

De même pour la diminution de w de 10 le déchiffrement ne fonctionne pas.

Tandis que lorsqu'on le fait varier juste autour de 39 Le déchiffrement peut bien fonctionner si la génération des clés marche ; par exemple pour $w = 35$ ou 43 ça fonctionne (pas toujours) mais ça prend plus de temps et plus d'itérations.

```
karim@PCKarim:/mnt/c/users/rahma/desktop$ ./MDPC3
Génération des clés:
Temps pris pour la Génération des clés est 0.333271:

Chiffrement:
Temps pris pour le chiffrement est 0.030745:

Déchiffrement:
BitFlipping réussie en 8 iterations
Temps pris pour le déchiffrement est 1.054703:

Le message est déchiffré avec succès
```

FIGURE 8 – Résultat d'application avec $w = 35$

La variation de w affecte le poids de nos clés les polynômes x et y , donc c'est tout a fait logique que notre algorithme ne marche pas si on effectue ces changement.